JPMorgan Midas Transaction-Processing Simulation

Kafka ingestion, validation, persistence, incentive integration, balance updates, and REST queries

Project context: JPMorgan Chase & Co. Advanced Software Engineering job simulation

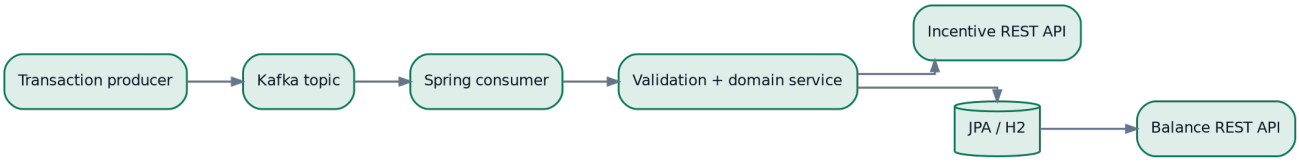
Status: Forage software-engineering simulation

A Java and Spring Boot training project centered on consuming transaction events from Kafka, validating and modeling them, persisting results with Spring Data JPA and H2, calling an external incentive REST API, updating user balances, and exposing balance data through a REST controller.

Technology stack

Java 17	**Spring Boot**	**Apache Kafka**	**Spring Data JPA**
H2	**RestTemplate**	**Maven**	**Embedded Kafka tests**

System at a glance



Event ingestion

Consumes transaction messages from a configurable Kafka topic.

Transaction workflow

Validates entities, calls the incentive service, and persists outcomes.

Balance management

Applies transaction effects to user account balances.

Verification

Uses Maven tests, embedded Kafka, H2, and debugger-driven inspection.

System design and component ownership

The simulation follows an event-driven backend shape. Kafka decouples event production from processing; the Spring service validates a transaction, enriches it through an external service, persists it, adjusts balances, and offers a separate HTTP read path.

Component	Responsibility
Kafka topic	Carries serialized transaction messages into the service.
Spring consumer	Deserializes and forwards events to domain processing.
Validation/domain layer	Checks transaction and user constraints before state changes.
Incentive client	Calls an external REST endpoint through RestTemplate.
JPA + H2	Persists entities and supports repeatable local testing.
Balance controller	Returns user-balance data as JSON over REST.

Key design decisions

- Use an event consumer for transaction intake and a REST endpoint for balance reads.
- Keep persistence behind JPA repositories and entity models.
- Test messaging with embedded Kafka rather than requiring an external broker.
- Use H2 to make the simulation self-contained and repeatable.

Core execution flow

A Kafka event enters the service, passes validation, receives an incentive calculation, is persisted, and updates the relevant user balance. A REST controller reads the final balance state independently.

Implementation notes

- The project is described as a completed Forage simulation rather than a deployed banking system.
- Transaction messages are deserialized from a configurable topic.
- An external incentive API is integrated into the transaction workflow.
- Entity persistence and balance updates are verified through local tests.
- Debugger inspection was used across messaging, database, and external API interactions.

Representative Spring processing structure

```
        @KafkaListener(topics = "${general.kafka-topic}")
        public void onTransaction(Transaction tx) {
            validate(tx);
            Incentive incentive = incentiveClient.calculate(tx);
            Transaction saved = transactionRepository.save(
                tx.withIncentive(incentive.amount())
            );
            userService.applyBalance(saved);
        }

        @GetMapping("/balance/{userId}")
        public BalanceResponse balance(@PathVariable long userId) {
            return balanceService.read(userId);
        }
```

Representative structure reconstructed from the completed simulation summary. The public repository snapshot currently contains only minimal starter documentation, so this block is not presented as a verbatim source file.

Engineering review and production path

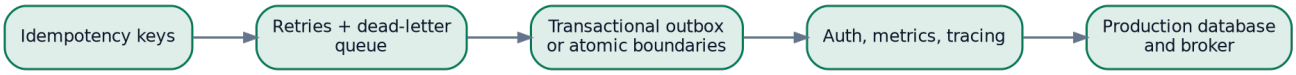
Engineering strengths

- Demonstrates event-driven Java backend concepts.
- Integrates messaging, persistence, HTTP, and an external service.
- Uses embedded infrastructure for repeatable tests.
- Shows debugger-led verification across system boundaries.

Current limitations

- The public repository snapshot exposes limited implementation evidence.
- H2 and embedded Kafka are simulation choices, not production infrastructure.
- Production concerns such as idempotency, dead-letter queues, authentication, and observability are outside the documented scope.
- No claim is made that this is an independent JPMorgan production system.

Recommended next steps



Why this is relevant to AI training work

- Provides realistic tasks for checking event ordering, validation, and balance correctness.
- Supports adversarial cases such as duplicate events, invalid users, and incentive-service failures.
- Demonstrates how to evaluate generated Java code across several system boundaries.
- Clearly distinguishes training-simulation evidence from production claims.

Repository:
https://github.com/SidheshwarSarangal/forage-midas

Scope and provenance:
Prepared from the project summary in the supplied resume and the public Forage repository metadata. The public repository currently contains a minimal starter snapshot, so representative architecture is labeled accordingly.